

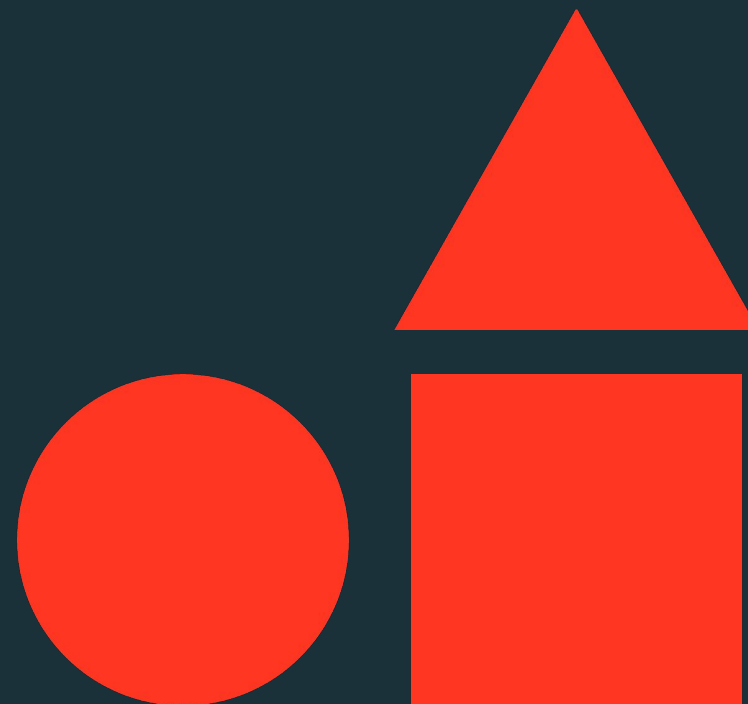


DATABRICKS SPECIALIST SESSIONS

Lakebase

Michele D'Addetta | Sr. Specialist Solutions Engineer

Matteo Monaldi | Sr. Specialist Solutions Engineer



Agenda

Introduction

Use Cases Patterns

Use Cases Qualification

Pricing Model

Cost monitoring

Architecture

Security

Authentication and Authorization

Demos

Product Walkthrough & UC Synchronization

Key Takeaways



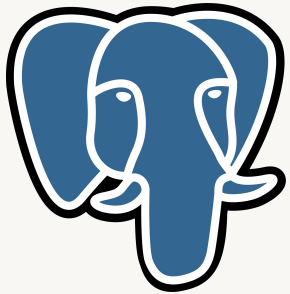
LAKEBASE

Introduction

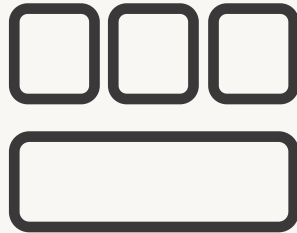


Lakebase

Fully-managed Postgres built for intelligent applications



Postgres database
engine



Separation of storage
and compute

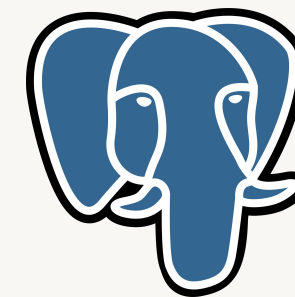


Integrated with the
Lakehouse



Postgres database engine

Fastest growing database engine of last decade



Postgres Engine

High perf: <10ms latency, >10ks QPS ✓

Full Postgres SQL engine: transactions (ACID), indexes, views, pk/fk enforcement, joins, CTEs, triggers, stored procedures ✓

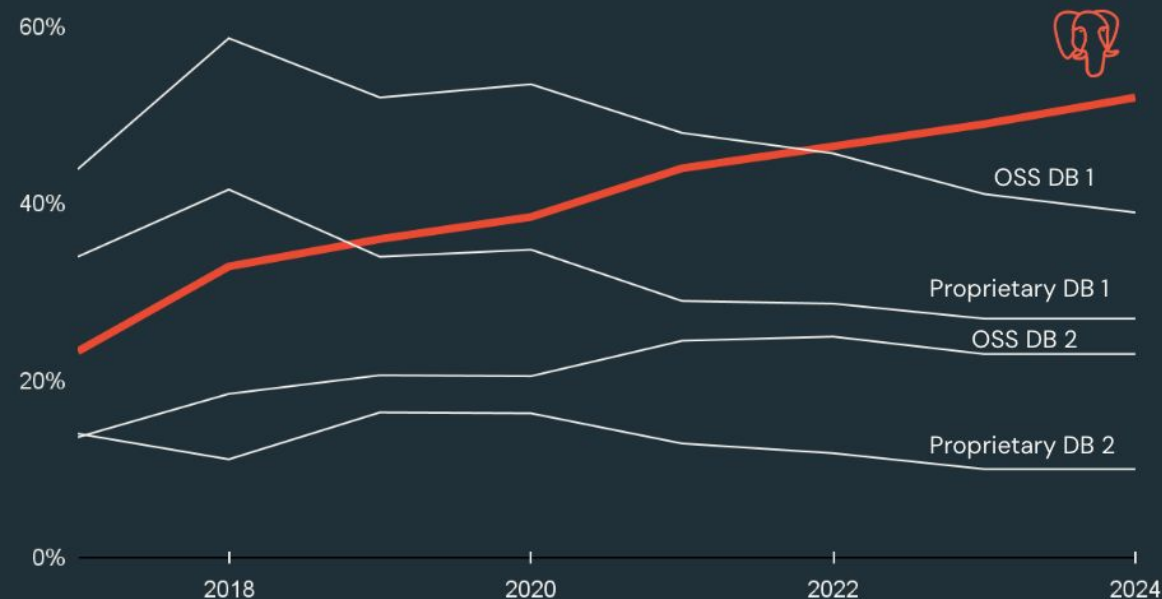
Rich ecosystem

Extensions: incl: PostGIS, pgvector ✓

Drivers: Java, Python, etc ✓

Tools: pgAdmin, DBeaver, PSQL ✓

StackOverflow Survey: Most Popular Databases



Separation of storage and compute

Fully managed, easy to use, and enterprise ready



Decoupled compute and storage

Independent compute/storage scaling ✓

Pause/resume provisioned compute ✓

Databricks Serverless Infra: hardened compute & encrypted storage ✓



High-availability and recovery

Multi High Availability Zone Failover ✓

Readable Secondaries ✓

Point-in-Time Recovery up to 35 days powered by copy-on-write clone ✓



Secure

Databricks Identity & OAuth ✓

Private Link / IP ACLs ✓

Transport Layer Security (TLS) encryption ✓



Management / observability

Built in monitoring ✓

Databricks Apps resource ✓

Terraform; Billing Tags ✓



Integrated with Lakehouse

Easy to use



Fully-Managed Data Synchronization

- Sync from Lakehouse table – Postgres ✓
- Sync from Postgres – Lakehouse table coming soon
- Snapshot, Triggered, Continuous ✓
- Multiple table syncs in a pipeline ✓



AI Platform: Online Feature Store

- High performance feature lookup ✓
- Feature store for real-time serving, with offline training data in Lakehouse ✓



Integrated with Unity Catalog

- Register Postgres database as Catalog ✓
- Governed by UC ✓
- Lakehouse Federation ✓



Query Editor and Schema Browser

- Native Postgres query editor ✓
- Browse Postgres schema ✓



LAKEBASE

Use Cases Patterns



Ideal Use Cases

1. Serve Analytical Data to Applications (reverse ETL)

Low-latency dashboards and operational stores backing internal or customer facing applications

Ex. 1) -commerce company serves recent order info. 2) Customer-facing dashboards for monitoring performance of digital promotions.

2. Support Model Serving and Feature Serving (reverse ETL)

Perform automatic feature store lookups for real-time ML use cases

Ex. 1) Assess customer likelihood to buy product. 2) Recommend next best action for call center agents.

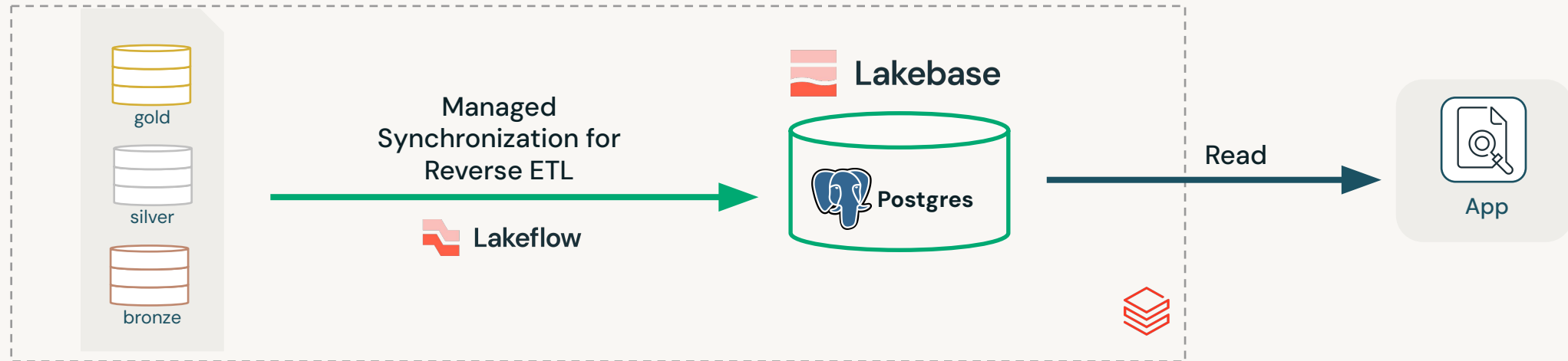
3. Store Application State

Ex. 1) Chatbot agent stores conversation history and customer preference. 2) Apps that needs a human review element to manage sign offs.



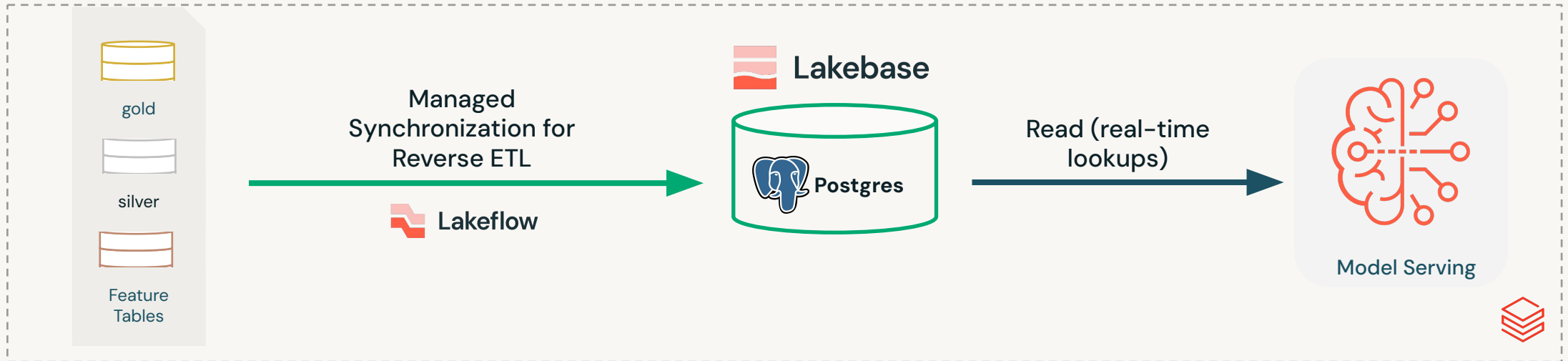
Reverse ETL (Apps)

Serving analytical data from the Lakehouse to applications



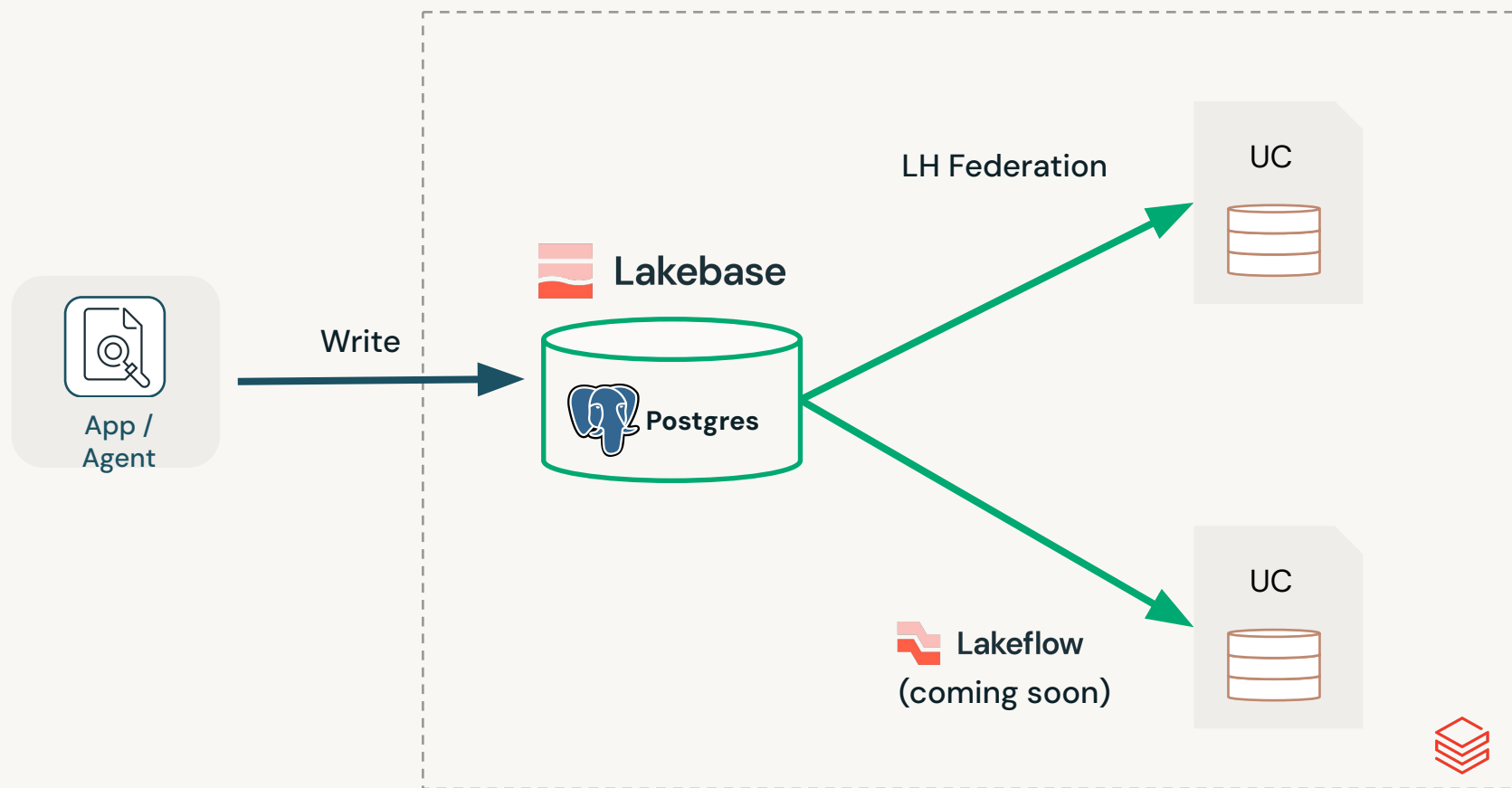
Reverse ETL (Feature Store)

Serve features to models for real-time lookups



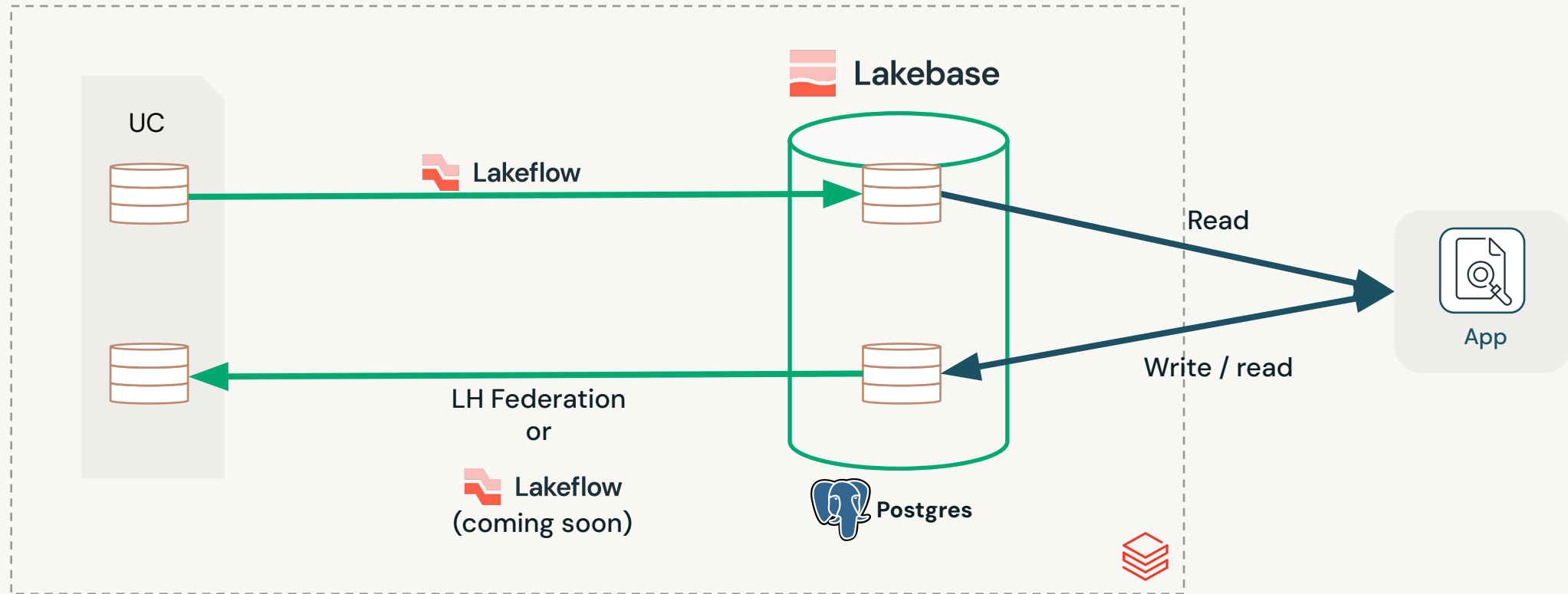
Application / Agent State Store

Write application state to Lakebase (and optionally use for analytics)



Reverse and forward ETL

Use analytical data in applications AND use operational data for analytics



Note that writes to a **particular** table are unidirectional



Sync from Lakehouse table – Postgres

Synchronization Options

Sync Mode	Description	Performance & Use Case
Snapshot	Runs once to take a complete copy of the source table into Postgres. Subsequent syncs replace all destination data atomically. Can be triggered manually, through API, or on a schedule.	Extremely efficient for one-off or infrequent bulk loads. 10x more performant versus other modes. Ideal when more than 10% of source data changes at a time.
Triggered	Starts with a snapshot, then only incremental changes since last sync are applied. Refreshes are manually triggered or scheduled.	Balances update lag and cost; fetches only changes since last run. Best for periodic updates. More frequent triggers may be more expensive than Continuous mode.
Continuous	Starts with a snapshot, then incrementally applies changes in real time as soon as they happen. No manual intervention needed.	Provides the lowest lag and most up-to-date data. Higher cost due to constant pipeline. Best for high-velocity use cases or applications demanding fresh data.



LAKEBASE

Use Case Qualification



Qualifying Databricks Options

If the workload is analytics then use DBSQL

Lakebase does NOT provide a low-latency alternative to DBSQL for analytics

OLTP

Goal: high-performance read/write of individual rows of data in **an application**

- High QPS (>10K), low latency <10ms
- Transactions, constraints etc.

OLAP

Goal: analyze aggregated data often at large scale **for analytics and/or BI**

- Aggregate queries over large volume of data often TBs
- Low concurrency needs

Qualification Questions

Question	Criteria
• What is overall data size and what is expected growth?	• 2 TB across all databases in an instance; This limit will be raised soon.
• What does update pattern look like? • What are your freshness expectations?	• Supports up to 10k rows / seconds • Lag between delta and Lakebase ~5+ seconds
• What are query patterns and performance needs?	• Point lookups and small range scans • Scans < 100k rows • Simple joins utilizing keys • QPS < 100k, ~10 ms latency



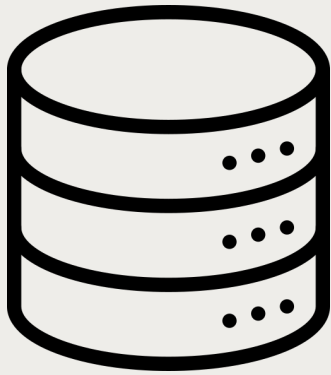
LAKEBASE

Pricing Model



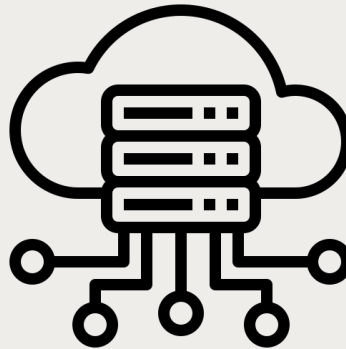
Pricing

Decoupled storage & compute, with provisioned compute



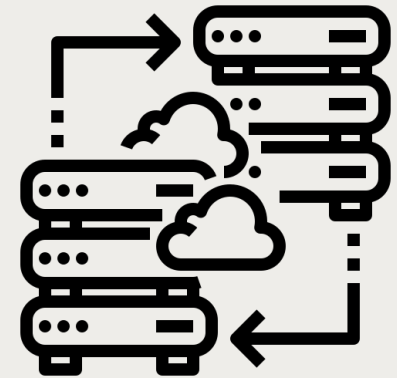
Storage

Databricks Storage
Unit (DSU) SKU



Compute

Database Serverless
Compute SKU



Data

Synchronization

Automated Serverless
Compute SKU

Pricing

Billing System Table

```
SELECT
    SUM(usage_quantity) AS total_usage_quantity,
    sku_name,
    usage_date
FROM system.billing.usage
WHERE billing_origin_product = 'DATABASE'
AND (
    sku_name LIKE '%DATABASE_SERVERLESS_COMPUTE%' OR
    sku_name LIKE '%DATABRICKS_STORAGE%' OR
    sku_name LIKE '%JOBS_SERVERLESS_COMPUTE%'
)
AND usage_date >= CURRENT_DATE - INTERVAL 1 DAY
GROUP BY sku_name, usage_date
ORDER BY usage_date
```



LAKEBASE

Architecture



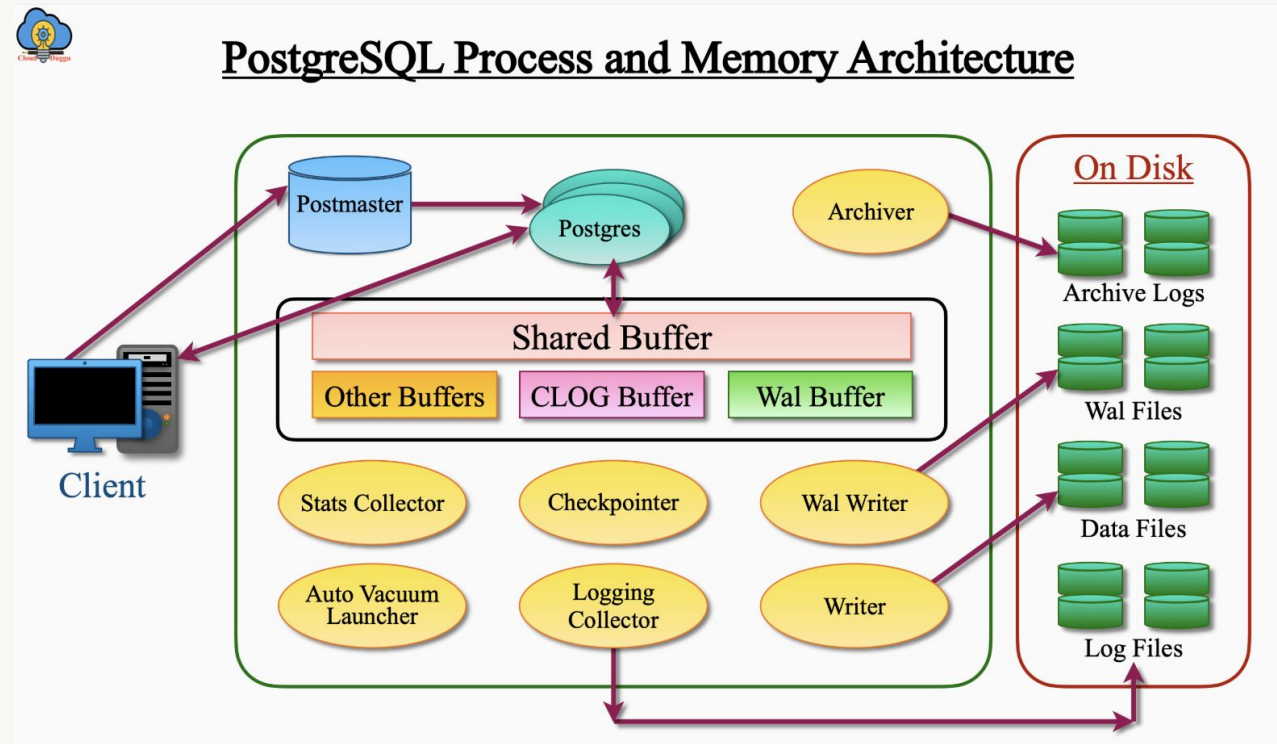
Traditional PostgreSQL

Monolithic Design

Compute, memory, background processes, storage all on one node

Benefits:

- Low latency, microsecond disk access
- Can co-locate app server and DB for faster interactions
- No cold starts



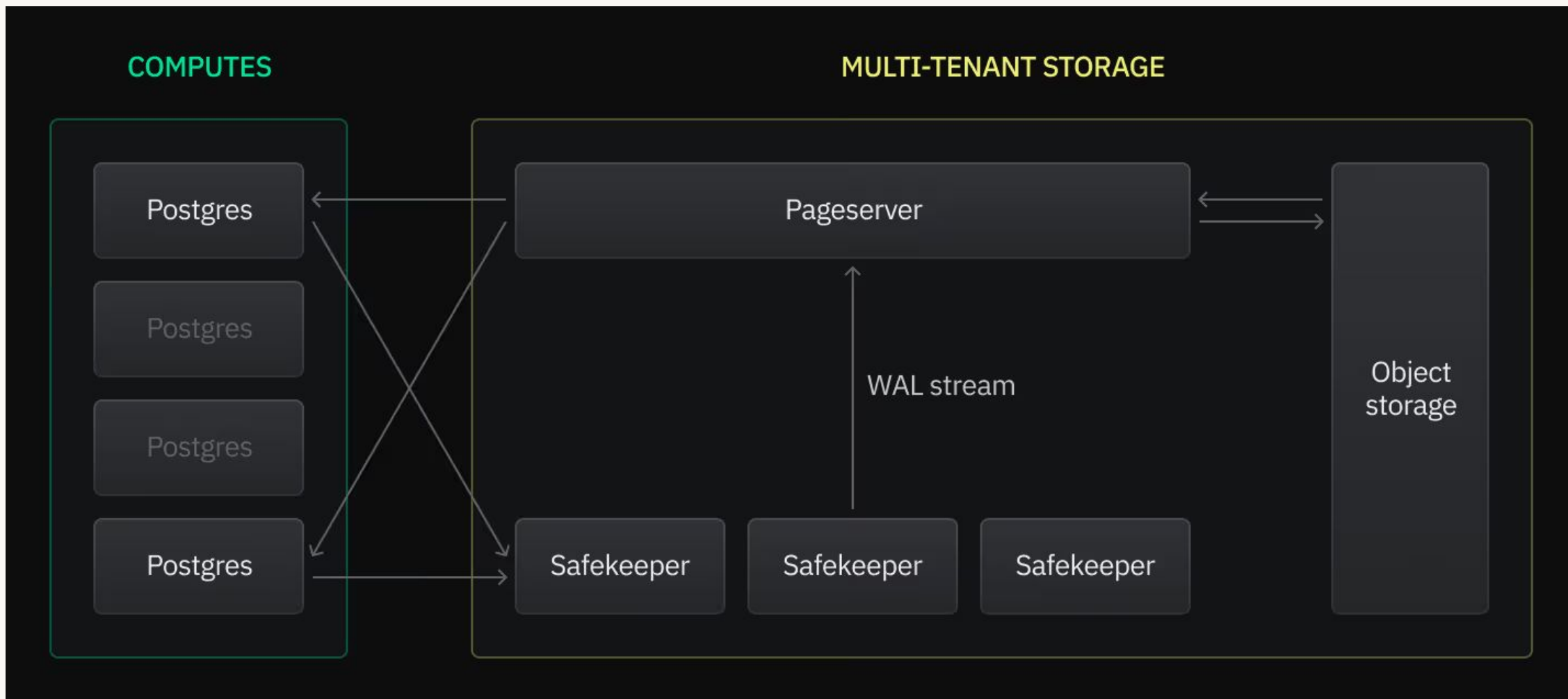
Traditional PostgreSQL

Limitations and risks

- **Single point of failure results in downtime or data loss**
 - Machine or datacenter failure
 - Disk failure
- **Scaling and Performance Limits**
 - Inefficient resource utilization across storage/compute
 - Inefficient resource utilization across time (peak times)
- **Recovery and Resiliency**
 - Slow restores from backups (dependent on storage size)
 - HA/DR complex to set up and maintain
- **High operational overhead**
 - Capacity planning, backups, replication, failover, manual recovery, etc.

Lakebase Architecture

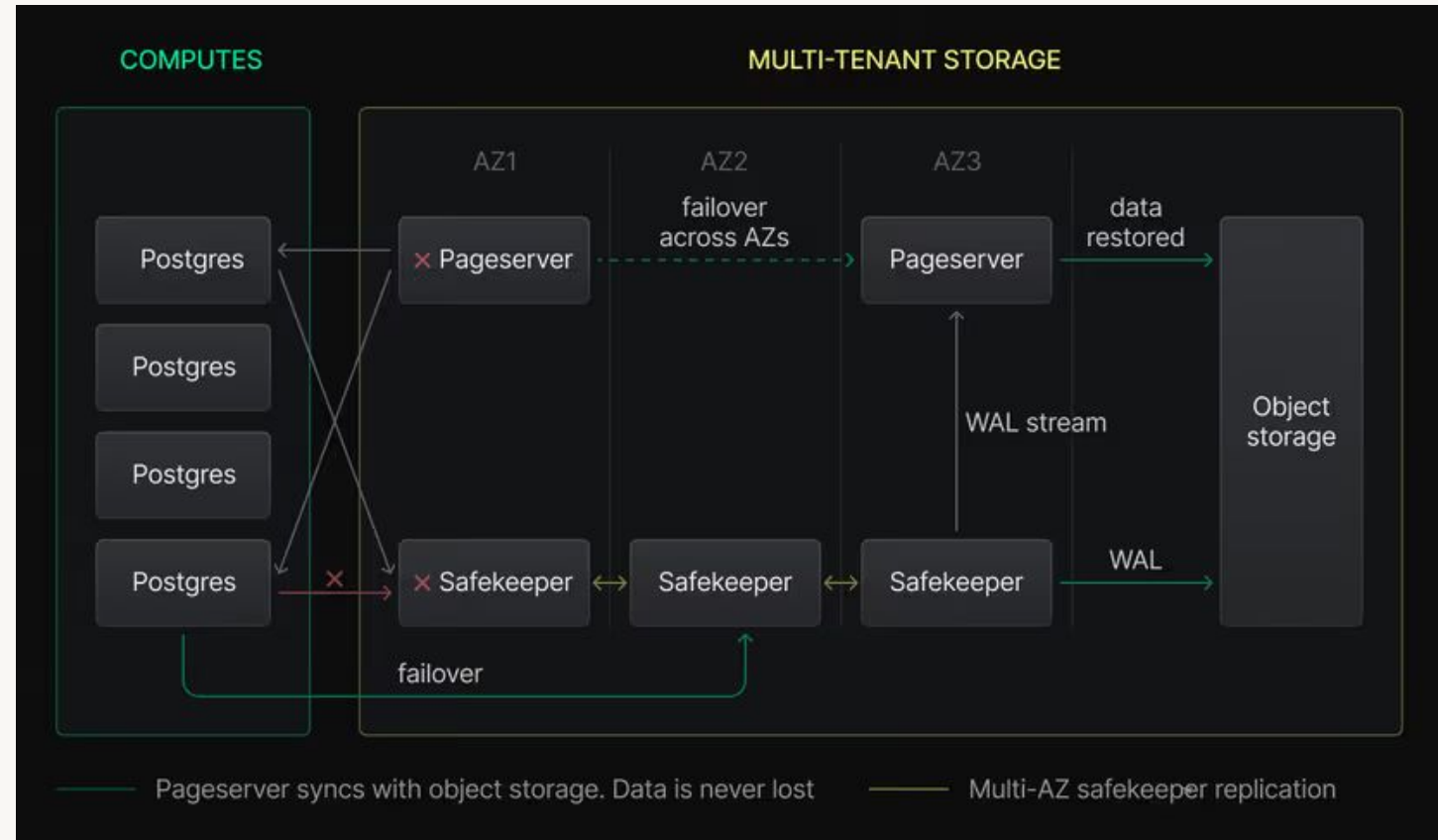
Decoupled compute and storage



High Availability and Read Replicas

Stateless compute simplifies availability

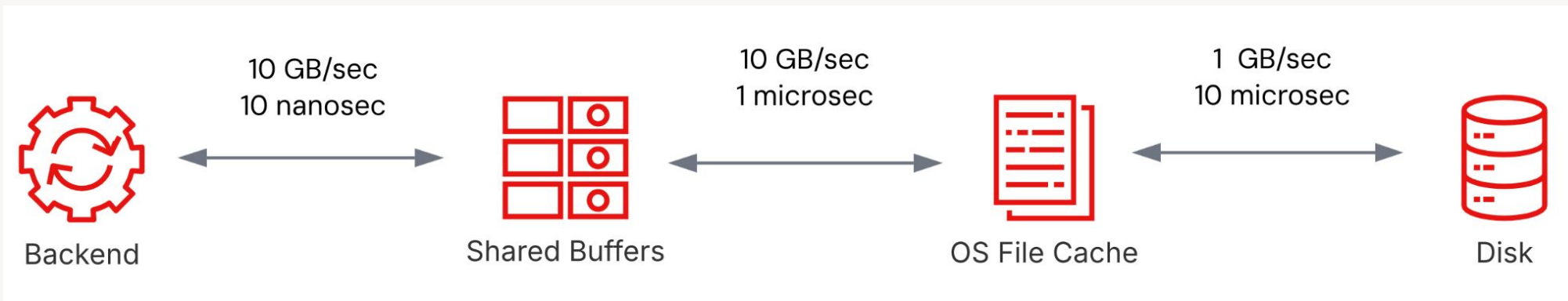
- **Compute failover**
 - Seconds to mins depending on location of failure
- **Storage redundancy across AZs**
 - Safekeepers replication
 - Pageserver failover
 - Object storage: 99.9999999999%
- **Read replicas perform read operations on same data**
 - No replication or duplication



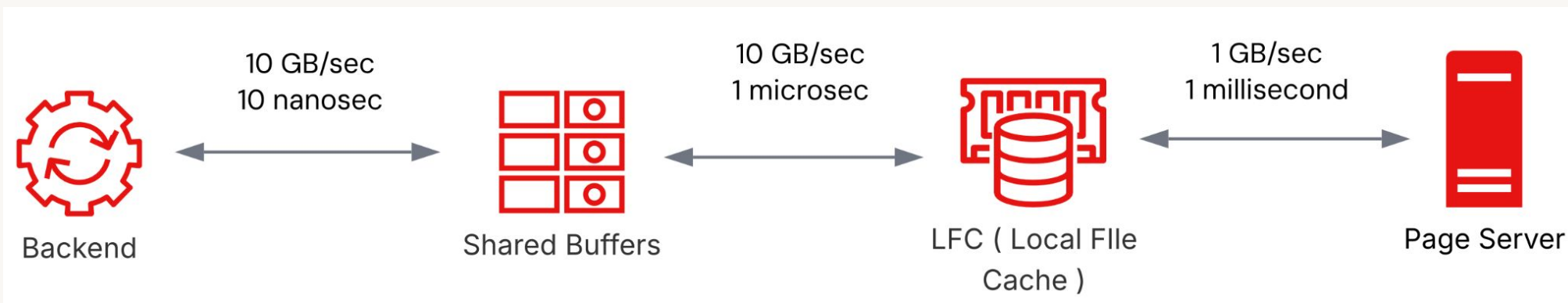
Caching in Lakebase

Local File Cache

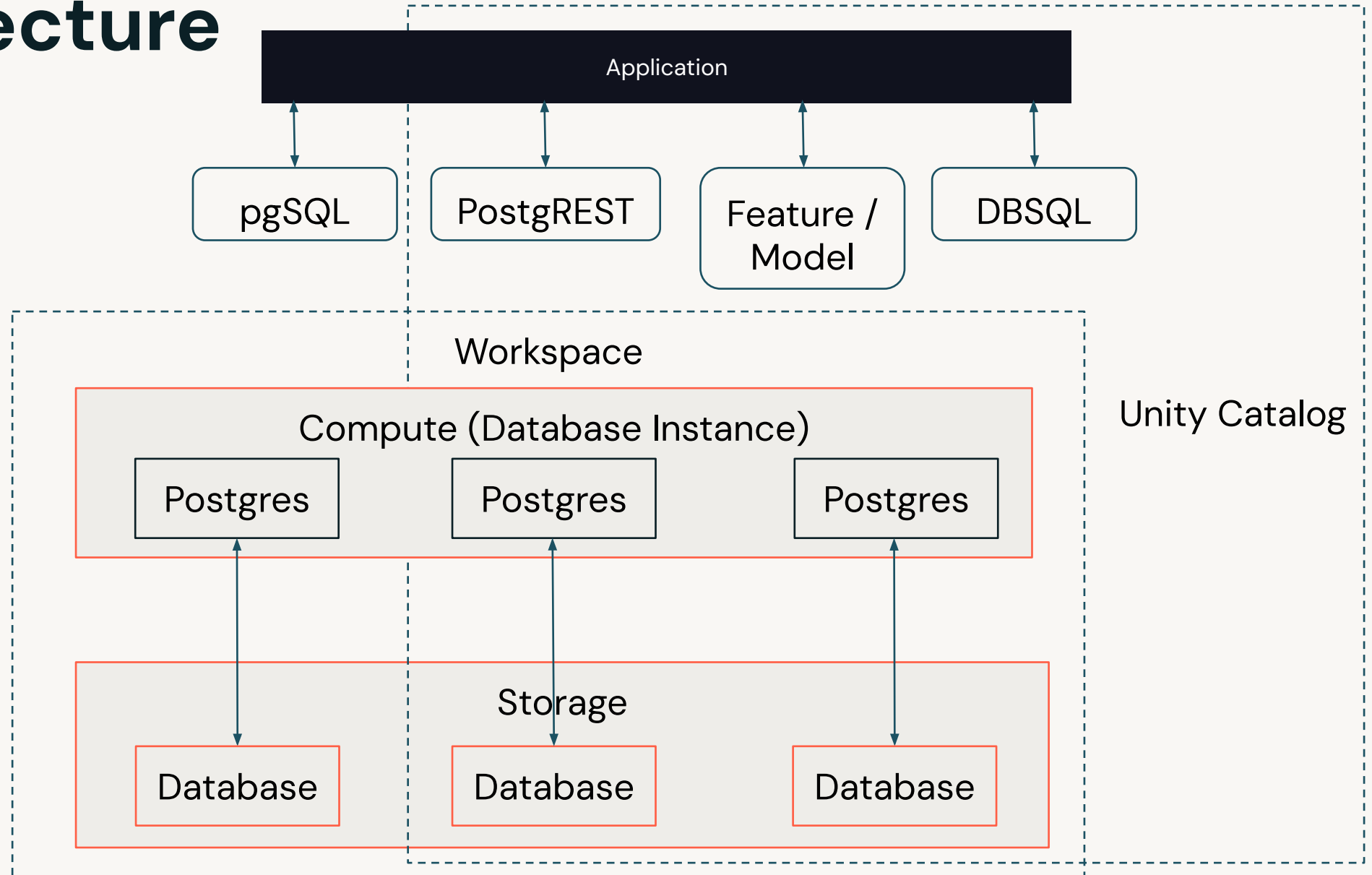
Traditional postgres



Lakebase



Architecture



Point In Time Recovery (PITR)

Based on copy-on-write

- Copy on Writes minimizes extra storage and leaves shared data unmodified
- Can create child instance in Lakebase (PITR) based on WAL



LAKEBASE

Neon Features



Branching

Based on copy-on-write

- Create branches for development, testing, temporary environments, etc.



Autoscaling and Scale-to-zero

Virtue of decoupled compute and storage



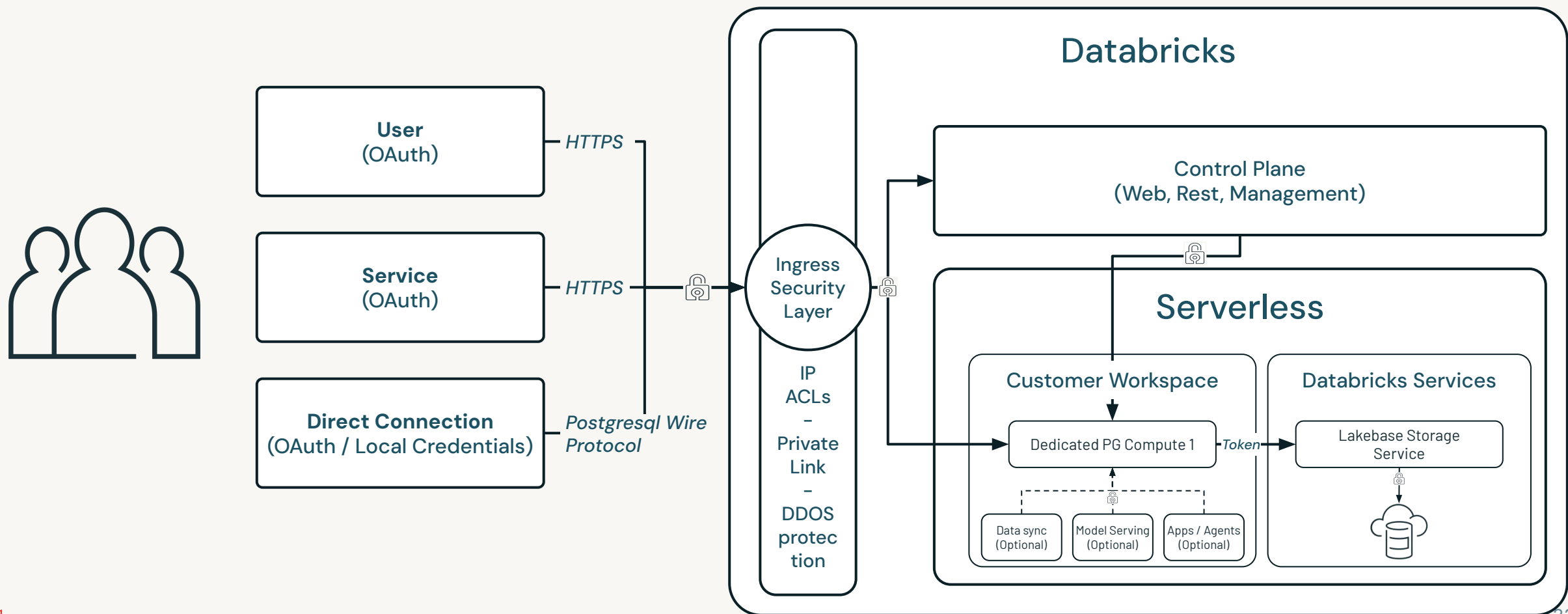
- Can vertically scale compute without scaling storage
- Lakebase today supports fixed capacity units (ie. compute does not autoscale)
- Neon supports auto-scaling and scale-to-zero, with auto-termination in case of inactivity (coming in Lakebase)

LAKEBASE

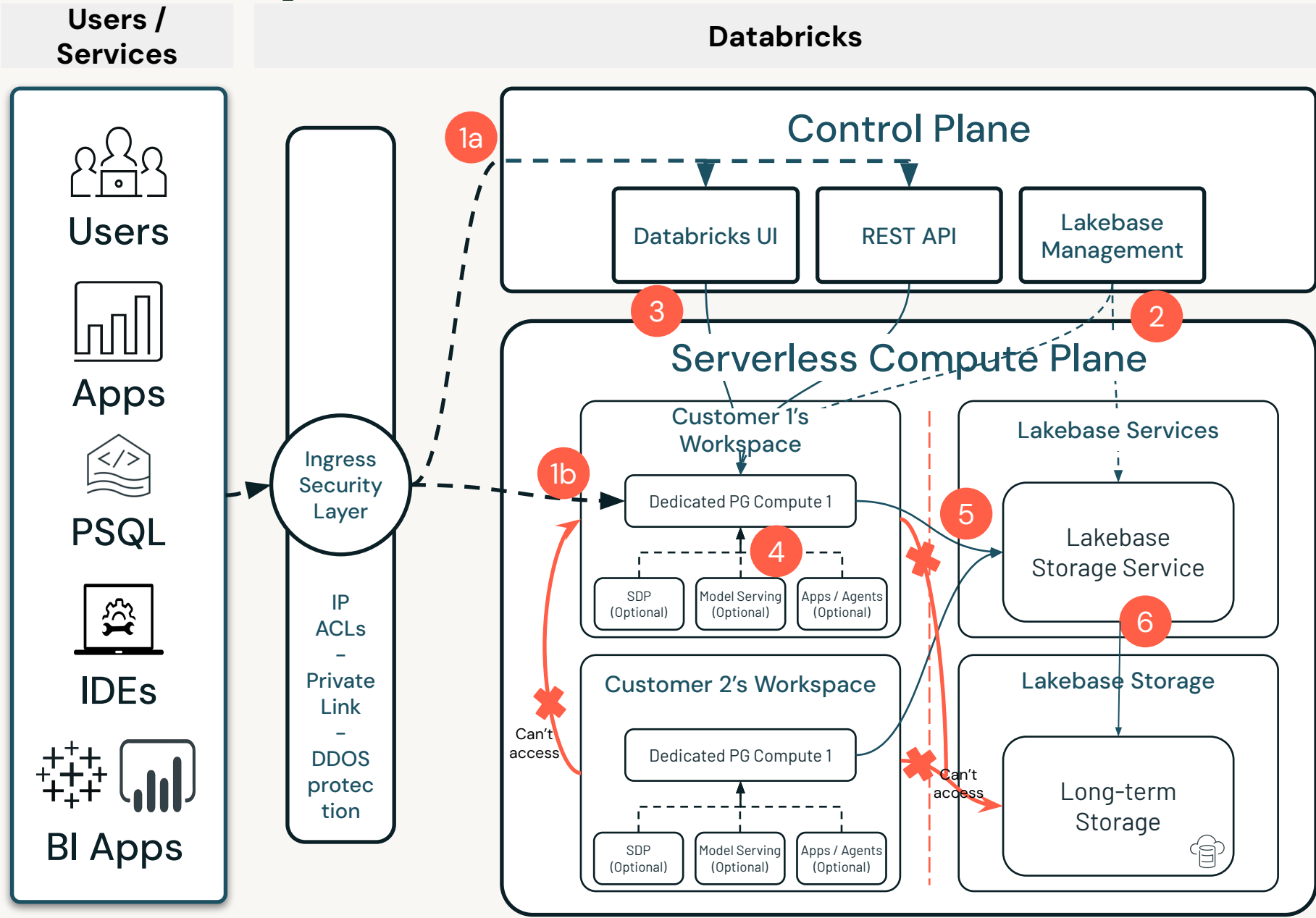
Security



Lakebase Connectivity



Lakebase Deep Dive



LAKEBASE

Authentication and Authorization



Lakebase Authentication

Lakebase provides PostgreSQL native and Workspace authentication options

PostgreSQL Authentication



PostgreSQL Lakebase account authentication

- Workspace user credentials (OAuth)
- PostgreSQL login

Databricks managed Authentication



Databricks Service Authentication

- Workspace user authentication (OAuth)
- DB managed OAuth M2M authentication

Lakebase Authorization

Lakebase provides PostgreSQL native and Unity Catalog authorization options

PostgreSQL Authorization



PostgreSQL Lakebase account authorization

- Native permissions and privileges

Unity Catalog Governance



Unified Authorization

- UC-managed governance
- Native services such as Notebooks, REST API, DB SQL, PostgREST API, Model Serving and Apps

Read/Write Postgres tables

Access approach	Endpoint	Authentication	Access mode	Latency
PostgREST	Online table direct URL	● OAuth token from UI ● OAuth token with SP	Read	Lowest
Postgres SQL	Postgres Host URL	● OAuth token from UI ● OAuth token from workspace ● OAuth token with SP ● PAT token	Read/Write	Low



LAKEBASE

Demo



LAKEBASE

Key Takeaways



Lakebase

Key Takeaways

- Managed Postgres, natively integrated with Databricks Lakehouse
- Decoupled compute & storage
- Seamless data sync
- Secure by design
- Optimized for modern use cases



